



LECTURE 7: GETTING STARTED WITH XV6 PROCESS & CODE

Code Organization of Xv6

File	Description
bio.c	Disk block cache for the file system.
console.c	Connect to the user keyboard and screen.
entry.S	Very first boot instructions.
exec.c	exec() system call.
file.c	File descriptor support.
fs.c	File system.
kalloc.c	Physical page allocator.
kernelvec.S	Handle traps from kernel.
log.c	File system logging and crash recovery.
main.c	Control initialization of other modules during boot.
pipe.c	Pipes.
plic.c	RISC-V interrupt controller.
printf.c	Formatted output to the console.
proc.c	Processes and scheduling.
sleeplock.c	Locks that yield the CPU.
spinlock.c	Locks that don't yield the CPU.
start.c	Early machine-mode boot code.
string.c	C string and byte-array library.
swtch.S	Thread switching.
syscall.c	Dispatch system calls to handling function.
sysfile.c	File-related system calls.
sysproc.c	Process-related system calls.
trampoline.S	Assembly code to switch between user and kernel.
trap.c	C code to handle and return from traps and interrupts.
uart.c	Serial-port console device driver.
virtio_disk.c	Disk device driver.
vm.c	Manage page tables and address spaces.

- The xv6 kernel source is in the kernel/ sub-directory.
- The source is divided into files, following a rough notion of modularity;
- Table lists the files.
- The inter-module interfaces are defined in defs.h (kernel/defs.h).

Process Overview

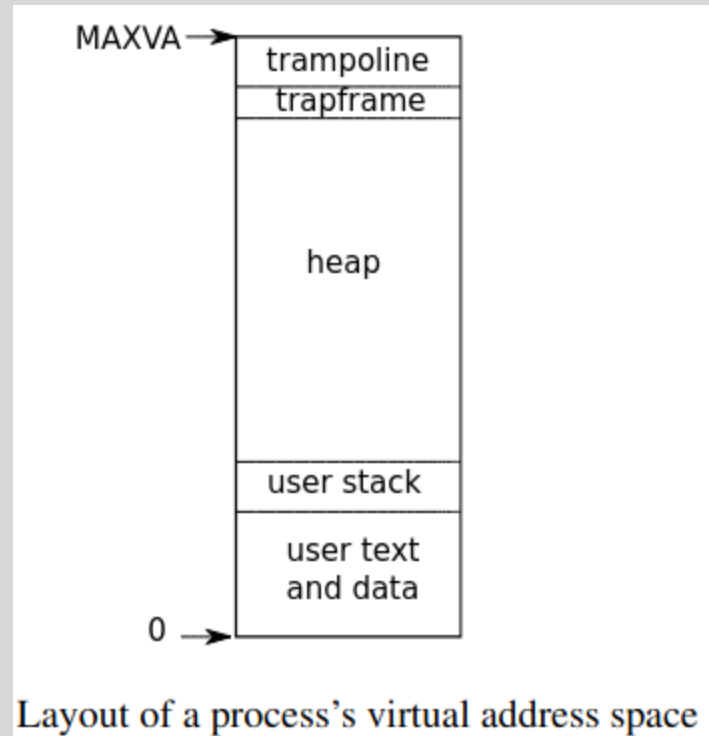
- The process abstraction prevents one process from wrecking or spying on another process's memory, CPU, file descriptors, etc.
- It also prevents a process from wrecking the kernel itself, so that a process can't subvert the kernel's isolation mechanisms.
- The kernel must implement the process abstraction with care because a buggy or malicious application may trick the kernel or hardware into doing something bad (e.g., circumventing isolation).
- The mechanisms used by the kernel to implement processes include the user/supervisor mode flag, address spaces, and time-slicing of threads.

Process Abstraction

- To help enforce isolation, the process abstraction provides the illusion to a program that it has its own private machine.
- A process provides a program with what appears to be a private memory system, or address space, which other processes cannot read or write.
- A process also provides the program with what appears to be its own CPU to execute the program's instructions.
- Xv6 uses page tables (which are implemented by hardware) to give each process its own address space.
- The RISC-V page table translates (or “maps”) a virtual address (the address that an RISC-V instruction manipulates) to a physical address (an address that the CPU sends to main memory)

Process Address Space

- Xv6 uses page tables (which are implemented by hardware) to give each process its own address space.
- The RISC-V page table translates (or “maps”) a virtual address (the address that an RISC-V instruction manipulates) to a physical address (an address that the CPU sends to main memory)Xv6 maintains a separate page table for each process that defines that process’s address space.
- As illustrated in Figure, an address space includes the process’s user memory starting at virtual address zero



- Instructions come first, followed by global variables, then the stack, and finally a “heap” area (for malloc) that the process can expand as needed.
- There are a few factors that limit the maximum size of a process’s address space: pointers on the RISC-V are 64 bits wide; the hardware uses only the low 39 bits when looking up virtual addresses in page tables; and xv6 uses only 38 of those 39 bits.
- Thus, the maximum address is $2^{(38 - 1)} = 0x3fffffff$, which is MAXVA

PCB in Xv6: struct proc

The central data structure in xv6 that represents a running process.

- **Synchronization:** A spinlock for safe, concurrent access.
- **State & Lifecycle:** state, killed, chan, xstate.
- **Identifiers:** pid (Process ID) and parent pointer.
- **Memory:** kstack (kernel stack), sz (memory size), and pagetable (user page table).
- **Context:** trapframe and context for managing execution and switching.
- **Resources:** ofile (open files) and cwd (current working directory).
- **Debugging:** name for process identification.
- **Summary of struct proc:** To store all the necessary information for the kernel to manage a process, schedule its execution, and handle its resources and state transitions.

```
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on
    chan
    int killed;                     // If non-zero, have been
    killed
    int xstate;                     // Exit status to be
    returned to parent's wait
    int pid;                         // Process ID

    // wait_lock must be held when using this:
    struct proc *parent;            // Parent process

    // these are private to the process, so p->lock need not
    be held.
    uint64 kstack;                  // Virtual address of kernel
    stack
    uint64 sz;                       // Size of process memory
    (bytes)
    pagetable_t pagetable;          // User page table
    struct trapframe *trapframe;    // data page for
    trampoline.S
    struct context context;          // swtch() here to run
    process
    struct file *ofile[NOFILE];     // Open files
    struct inode *cwd;              // Current directory
    char name[16];                  // Process name (debugging)
};
```

Threads & Context Switching

- **Thread of Control:** Each process has a **thread** that holds the state needed to execute it. This state includes its registers and stack contents.
- **Execution States:** A thread is either **running** on a CPU or **suspended** (not running, but ready to resume).
- **Context Switching:** The kernel switches between processes by suspending the current thread, **saving its state**, and then **restoring the saved state** of another thread to resume its execution.

User & Kernel Stacks

- **Two Stacks per Process:** Every process has two separate stacks: a **user stack** and a **kernel stack**.
- **User Stack:** Used when the process is executing its own user-level code.
- **Kernel Stack:** Used when the process enters the kernel (e.g., for a system call or interrupt).
- **Why two stacks?** The kernel stack is separate and **protected** from user code. This design ensures that a malicious or buggy user program cannot corrupt the kernel's stack, which could lead to a system crash.

System Calls and Traps

- **System Call Initiation:** A user process executes the `ecall` instruction to make a system call. This instruction serves as a "trap" into the kernel.
- **Kernel Entry:**
 - The `ecall` instruction automatically raises the CPU's privilege level to **Supervisor mode**.
 - It redirects the program counter to a pre-defined kernel entry point.
 - The kernel then switches from the **user stack** to the protected **kernel stack** to execute the system call's code.

Returning from the Kernel

- When the system call completes, the kernel prepares to return to the user process.
- **Restoring Context:** The kernel switches back to the user stack.
- **The sret Instruction:** The kernel executes the sret instruction.
 - Lowers the CPU's privilege level back to **User mode**.
 - Resumes execution of user code at the instruction immediately following the original `ecall`.

Blocking for I/O

- **Blocking:** A process's thread can "block" in the kernel while waiting for an event, such as I/O completion.
- **Waiting State:** The kernel places the thread in a waiting state, allowing the CPU to execute other processes.
- **Resumption:** When the I/O operation is finished, the kernel resumes the thread from where it left off, allowing it to continue its execution.

Process State

- enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

p->state indicates whether the process is allocated, ready to run, currently running on a CPU, waiting for I/O, or exiting

Page Table

- `p->pagetable` holds the process's page table, in the format that the RISC-V hardware expects.
- Xv6 causes the paging hardware to use a process's `p->pagetable` when executing that process in user space.
- A process's page table also serves as the record of the addresses of the physical pages allocated to store the process's memory.

Starting Xv6

- When the RISC-V computer powers on
 - Initialization and the boot loader loads the xv6 kernel into memory.
- Then, in machine mode, the CPU executes Xv6 starting at `_entry`
- The RISC-V starts with paging hardware disabled: virtual addresses map directly to physical addresses.

```
# qemu -kernel loads the kernel at 0x80000000
# and causes each hart (i.e. CPU) to jump there.
# kernel.ld causes the following code to
# be placed at 0x80000000.
.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + ((hartid + 1) * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start

spin:
    j spin
```

Boot Loader

- The loader loads the xv6 kernel into memory at physical address 0x80000000.
- The reason it places the kernel at 0x80000000 rather than 0x0 is because the address range 0x0:0x80000000 contains I/O devices

```
# qemu -kernel loads the kernel at 0x80000000
# and causes each hart (i.e. CPU) to jump there.
# kernel.ld causes the following code to
# be placed at 0x80000000.
.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + ((hartid + 1) * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start

spin:
    j spin
```

Initial Stack Setup (**_entry**)

- **Goal:** Prepare the system for C code execution by setting up a stack.
- **Location:** The kernel starts at `0x80000000` in assembly code (`_entry`).
- **Method:**
- Each CPU (hart) gets its own 4096-byte stack.
- The code calculates a unique stack address for each CPU by adding an offset to the base of `stack0`.
- The `sp` (stack pointer) register is set to the top of this stack, since RISC-V stacks grow downwards.
- **Final Step:** The `_entry` code calls the C function `start()`, transitioning control to C.

```
# qemu -kernel loads the kernel at 0x80000000
# and causes each hart (i.e. CPU) to jump there.
# kernel.ld causes the following code to
# be placed at 0x80000000.
.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + ((hartid + 1) * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start

spin:
    j spin
```


Transition to Supervisor Mode (**start()**) function

- **Goal:** Switch from privileged Machine mode to Supervisor mode.
- **Key Registers:**
- **mstatus:** Configured to tell the system to enter Supervisor mode upon return.
- **mepc:** Set to the address of the `main()` function, which is the next destination.
- **Hardware Setup:**
- Paging is disabled for now (`w_satp(0)`).
- All interrupts and exceptions are delegated to Supervisor mode.
- Physical Memory Protection (PMP) is configured to give Supervisor mode full memory access.
- **Final Step:** The `mret` instruction is executed. This performs the mode switch and jumps to the `main()` function, which is the beginning of the core kernel code.

```
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Program Counter to main, for mret.
    // requires gcc -mcmode=medany
    w_mepc((uint64)main);

    // disable paging for now.
    w_satp(0);

    // delegate all interrupts and exceptions to supervisor mode.
    w_medeleg(0xffff);
    w_mideleg(0xffff);
    w_sie(r_sie() | SIE_SEIE | SIE_STIE);

    // configure Physical Memory Protection to give supervisor mode
    // access to all of physical memory.
    w_pmpaddr0(0x3fffffffffffffff);
    w_pmpcfg0(0xf);

    // ask for clock interrupts.
    timerinit();

    // keep each CPU's hartid in its tp register, for cpuid().
    int id = r_mhartid();
    w_tp(id);

    // switch to supervisor mode and jump to main().
    asm volatile("mret");
}
```

main() Function - Core Initialization

- **Initialize all kernel components and start the first user process.**
- **Exclusive Execution: Only CPU 0 runs this initialization block to prevent conflicts.**
- **Initializing:**
 - **Console:** Enables screen output.
 - **Memory:** Sets up the physical page allocator and kernel page tables.
 - **Processes:** Creates the process table and the first user process.
 - **Traps & Interrupts:** Configures handlers for exceptions and device interrupts.
 - **File System:** Initializes the buffer cache, inode table, file table, and the virtual disk.
 - **Synchronization:** A global started flag ensures that other CPUs wait for the initialization to complete before they can start.

```
Void main()
{
    if(cpuid() == 0){
        consoleinit();
        printfinit();
        printf("\n");
        printf("xv6 kernel is booting\n");
        printf("\n");
        kinit();           // physical page allocator
        kvminit();         // create kernel page table
        kvmithart();       // turn on paging
        procinit();        // process table
        trapinit();        // trap vectors
        trapithart();      // install kernel trap
vector
        plicinit();        // set up interrupt
controller
        plicithart();     // ask PLIC for device
interrupts
        binit();          // buffer cache
        iinit();          // inode table
        fileinit();       // file table
        virtio_disk_init(); // emulated hard disk
        userinit();       // first user process
        __sync_synchronize();
        started = 1;
    }
}
```

userinit() - The First Process

- **Create the initial user-level process, called `init`.**
- **Steps:**
- `allocproc()`: A new process structure is allocated.
- `initproc` is assigned to point to this new process.
- The process's **current working directory** is set to the root (/).
- **Status:** The new process is marked as **RUNNABLE**, making it ready to be scheduled and executed by the CPU.
- **Next Step:** The kernel will load the `/init` program into this process's memory and then execute it, starting the user-space part of the operating system.

```
void
userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->cwd = namei("/");

    p->state = RUNNABLE;

    release(&p->lock);
}
```



Thank You